

LAPORAN PRAKTIKUM KUIS 2

SISTEM DETEKSI DAN REKOMENDASI BURNOUT MAHASISWA

MATA KULIAH SISTEM BERBASIS PENGETAHUAN



Disusun oleh Kelompok 5:

Alvi Choirinnikmah	2341760191
Ismi Atika	2341760036
Muhammad Eka Saputra	2341760021
Muhammad Toyfur D.R	2241760126

**JURUSAN TEKNOLOGI INFORMASI
PROGRAM STUDI D-IV SISTEM INFORMASI BISNIS
POLITEKNIK NEGERI MALANG
2026/2027**

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Burnout adalah kondisi kelelahan fisik dan mental yang cukup sering dialami oleh mahasiswa, terutama mahasiswa di bidang Teknologi Informasi. Hal ini biasanya disebabkan oleh banyaknya tugas, tekanan deadline, serta kebiasaan begadang untuk menyelesaikan tugas praktikum.

Mahasiswa Teknologi Informasi cenderung memiliki aktivitas belajar yang lebih intens, seperti coding, pengerjaan project, dan praktikum. Kondisi tersebut membuat risiko burnout menjadi lebih tinggi dibandingkan bidang lain. Oleh karena itu, diperlukan sebuah sistem yang dapat membantu mendeteksi tingkat burnout sejak dini serta memberikan rekomendasi penanganannya.

Pada praktikum ini, akan dilakukan analisis beberapa metode, yaitu SAW dan EDAS.

1.2 Rumusan Masalah

1. Bagaimana cara mendeteksi tingkat burnout mahasiswa menggunakan data mental health mahasiswa?
2. Bagaimana penerapan metode SAW dalam menentukan tingkat burnout mahasiswa?
3. Bagaimana penerapan metode EDAS dalam menganalisis tingkat burnout mahasiswa?
4. Bagaimana penerapan metode AHP dan TOPSIS dalam menganalisis tingkat burnout mahasiswa?
5. Bagaimana hasil perbandingan metode SAW, EDAS, AHP, dan TOPSIS dalam memberikan rekomendasi tingkat burnout mahasiswa?

1.3 Tujuan

1. Mendeteksi tingkat burnout mahasiswa berdasarkan data mental health dan aktivitas akademik.
2. Mengimplementasikan metode SAW untuk menghitung tingkat burnout mahasiswa.
3. Mengimplementasikan metode EDAS untuk melakukan analisis tingkat burnout mahasiswa.
4. Mengimplementasikan metode AHP dan TOPSIS untuk melakukan analisis tingkat burnout mahasiswa.
5. Membandingkan hasil metode SAW, EDAS, AHP, dan TOPSIS untuk mengetahui metode yang lebih efektif dalam proses pengambilan keputusan.

BAB 2 DATASET & PREPROCESSING

2.1 Deskripsi Dataset

Dataset yang digunakan adalah *Student Mental Health and Burnout Dataset* dari Kaggle. Dataset ini berisi informasi terkait kondisi akademik, psikologis, dan gaya hidup siswa. Data ini berfungsi sebagai basis pengetahuan (*Knowledge Base*) awal untuk mengidentifikasi pola-pola yang memicu burnout pada mahasiswa.

Atribut	Keterangan
Sumber	Kaggle - Student Mental Health and Burnout Dataset
Jumlah Data	150.000 baris
Jumlah Fitur	20 kolom (termasuk ID dan label target)
Label Target	Burnout_level

2.2 Variabel Penelitian

Pada penelitian ini digunakan 10 variabel utama yang dianggap relevan terhadap tingkat burnout mahasiswa.

Kode	Variabel	Jenis
C1	Variabel Level	Cost
C2	Anxiety Score	Cost
C3	Depression Score	Cost
C4	Academic Pressure	Cost
C5	Social Support	Benefit
C6	Screen Time	Cost
C7	Sleep Quality	Benefit
C8	Daily Sleep Hours	Benefit
C9	Coding Hours	Cost
C10	Assignment Deadline Pressure	Cost

Keterangan:

- a. Benefit = semakin besar nilainya semakin baik
- b. Cost = semakin kecil nilainya semakin baik.

2.3 Persiapan Data

2.3.1 Akuisisi Data

1. Import Library

Library yang digunakan dalam praktikum ini adalah pandas, numpy, matplotlib, dan sklearn.preprocessing

2. Load Dataset

Dataset dibaca menggunakan fungsi `read_csv()` dan library pandas. Dataset yang digunakan adalah Student Mental Health Burnout yang berisi data kesehatan mental mahasiswa. Setelah dibaca, spasi tersembunyi pada nama kolom dihapus menggunakan `.str.strip()` untuk menghindari error saat pemanggilan kolom.

3. Pengecekan Dataset dan RAW data

Dilakukan pengecekan awal terhadap dataset sebelum ke tahap preprocessing meliputi jumlah data, informasi dataset, nama kolom, missing value, dan statistik deskriptif.

```
# =====  
# IMPORT LIBRARY  
# =====  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
from sklearn.preprocessing import LabelEncoder  
from sklearn.preprocessing import StandardScaler  
  
# =====  
# LOAD DATASET  
# =====  
  
df = pd.read_csv('/content/student_mental_health_burnout.csv')  
  
# menghapus spasi tersembunyi  
df.columns = df.columns.str.strip()  
  
# =====  
# MENAMPILKAN RAW DATA  
# =====  
print("Jumlah Data :", df.shape)  
print("\nInformasi Dataset")  
  
print("\ns Data Pertama")  
display(df.head())
```

```
# =====  
# INFORMASI DATASET  
# =====  
  
print(df.info())  
  
print("\nNama Kolom Dataset")  
print(df.columns.tolist())  
  
print("\nMissing Value")  
print(df.isnull().sum())  
  
print("\nStatistik Deskriptif")  
display(df.describe(include='all'))
```

2.3.2 Pra-pemrosesan Data

Sebelum data digunakan dalam proses analisis, dilakukan preprocessing data agar kualitas data menjadi lebih baik. Tahapan preprocessing yang dilakukan meliputi menghapus duplikat, mengisi missing values numerik dan kategorikal.

```

# =====
# PREPROCESSING DATA
# =====

print("\n===== PREPROCESSING DATA =====\n")

# 1. Hapus data duplikat
df = df.drop_duplicates()

# 2. Menangani missing value
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].mean())

categorical_cols = df.select_dtypes(include=['object']).columns

for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

print("\nPreprocessing selesai")
display(df.head())

```

2.3.3 Penambahan Variabel Baru

Karena dataset asli tidak memiliki variabel `coding_hours_per_day` dan `assignment_deadline_pressure`, maka dilakukan penambahan variabel simulasi menggunakan random number. Variabel tambahan `coding_hours_per_day` dan `assignment_deadline_pressure`. Penambahan variabel dilakukan menggunakan library NumPy agar data dapat digunakan dalam proses perhitungan SAW, EDAS, AHP, dan TOPSIS.

```

# =====
# MENAMBAHKAN VARIABEL BARU
# =====

# agar hasil random tetap sama
np.random.seed(42)

# =====
# CODING HOURS PER DAY
# =====

df['coding_hours_per_day'] = np.random.uniform(
    1,
    10,
    size=len(df)
)

# =====
# ASSIGNMENT DEADLINE PRESSURE
# =====

df['assignment_deadline_pressure'] = np.random.randint(
    1,
    11,
    size=len(df)
)

display(
    df[['coding_hours_per_day',
         'assignment_deadline_pressure']]
    .head()
)

```

2.3.4 Konversi Data Kategorikal

Perhitungan yang dilakukan hanya dapat diproses dalam bentuk numerik. Oleh karena itu, variabel kategorikal harus dikonversi ke bentuk numerik sebelum digunakan dalam perhitungan.

a. Konversi Stress Level

Metode `stress_level` memiliki 3 kategori yang mempresentasikan stress mahasiswa. Konversi dilakukan secara ordinal karena terdapat urutan tingkatan yang jelas.

```
1) # =====  
# KONVERSI STRESS LEVEL  
# =====  
  
stress_mapping = {  
    'Low': 1,  
    'Medium': 2,  
    'High': 3  
}  
  
df['stress_level_num'] = (  
    df['stress_level']  
    .map(stress_mapping)  
)  
  
display(  
    df[[  
        'stress_level',  
        'stress_level_num'  
    ]].head()  
)  
  
***  
stress_level stress_level_num  
0      High      3  
1      High      3  
2      Low       1  
3      High      3  
4      High      3
```

b. Konversi Sleep Quality

Variabel `sleep_quality` memiliki 3 kategori yang mempresentasikan kualitas tidur mahasiswa. Sama seperti `stress level`, konversi dilakukan secara ordinal karena terdapat urutan kualitas yang jelas.

```
# =====  
# KONVERSI SLEEP QUALITY  
# =====  
  
print(df['sleep_quality'].unique())  
  
# =====  
# MAPING SLEEP QUALITY  
# =====  
  
sleep_mapping = {  
    'Poor': 1,  
    'Average': 2,  
    'Good': 3  
}  
  
df['sleep_quality_num'] = (  
    df['sleep_quality']  
    .map(sleep_mapping)  
)  
  
display(  
    df[[  
        'sleep_quality',  
        'sleep_quality_num'  
    ]].head()  
)  
  
***  
['Average' 'Poor' 'Good']  
sleep_quality sleep_quality_num  
0      Average      2  
1      Poor       1  
2      Good       3  
3      Poor       1  
4      Good       3
```

BAB 3 IMPLEMENTASI MODEL

3.1 Metode SAW

Metode SAW merupakan metode pengambilan keputusan multikriteria dengan konsep penjumlahan terbobot. Metode ini digunakan untuk menentukan alternatif terbaik berdasarkan nilai preferensi tertinggi.

Google Collab :

<https://colab.research.google.com/drive/10wxW9670bQO8ausLqOmjCPNZfxfvK7ef?usp=sharing>

Tahapan metode SAW meliputi:

1. Membuat Data SAW

Proses pengambilan variabel yang digunakan dalam metode SAW. Data dipilih berdasarkan 10 kriteria yang mempengaruhi burnout mahasiswa sebagai atribut penilaian.

```
MEMBUAT DATA SAW

# =====
# DATA SAW
# =====
# mengambil 10 data pertama

data = {
    'Alternatif': df['student_id'].head(10),
    'C1 (Stress Level)': df['stress_level_num'].head(10),
    'C2 (Anxiety Score)': df['anxiety_score'].head(10),
    'C3 (Depression Score)': df['depression_score'].head(10),
    'C4 (Academic Pressure)': df['academic_pressure_score'].head(10),
    'C5 (Social Support)': df['social_support_score'].head(10),
    'C6 (Screen Time)': df['screen_time_hours'].head(10),
    'C7 (Sleep Quality)': df['sleep_quality_num'].head(10),
    'C8 (Daily Sleep Hours)': df['daily_sleep_hours'].head(10),
    'C9 (Coding Hours)': df['coding_hours_per_day'].head(10),
    'C10 (Assignment Deadline)': df['assignment_deadline_pressure'].head(10)
}

df_saw = pd.DataFrame(data)
print("=== DATA SAW ===")
display(df_saw)
```

	Alternatif	C1 (Stress Level)	C2 (Anxiety Score)	C3 (Depression Score)	C4 (Academic Pressure)	C5 (Social Support)	C6 (Screen Time)	C7 (Sleep Quality)	C8 (Daily Sleep Hours)	C9 (Coding Hours)	C10 (Assignment Deadline)	
0	100001	3	10	3	4	6	6.1	2	6.8	4.370861	3	
1	100002	3	2	10	8	9	3.0	1	4.7	9.556429	6	
2	100003	1	2	7	8	3	1.5	3	4.8	7.587945	2	
3	100004	3	3	3	4	9	7.0	1	6.7	6.387926	8	
4	100005	3	7	7	6	4	5.4	3	6.7	2.404168	4	
5	100006	3	8	4	9	2	4.1	1	4.8	2.403951	10	
6	100007	1	5	9	9	6	7.8	2	8.0	1.522753	8	
7	100008	2	5	10	3	6	7.6	1	6.3	8.795585	8	
8	100009	4	4	4	7	4	4.3	3	6.4	6.440002	8	

2. Menentukan Bobot

Setiap kriteria diberikan bobot yang sama yaitu 0.10 sehingga total bobot = 1.0.

```
BOBOT KRITERIA

# =====
# BOBOT KRITERIA
# =====

weights = np.array([
    0.10, # stress
    0.10, # anxiety
    0.10, # depression
    0.10, # academic pressure
    0.10, # social support
    0.10, # screen time
    0.10, # sleep quality
    0.10, # sleep hours
    0.10, # coding hours
    0.10 # assignment deadline
])

print("Total Bobot =", weights.sum())

... Total Bobot = 1.0
```

3. Atribut Kriteria

Setiap kriteria juga didefinisikan tipenya benefit dan cost.

- a. Benefit : semakin tinggi nilainya semakin baik
(C5_Social Support, C7_Sleep Quality, C8_Daily Sleep Hours)
- b. Cost : semakin rendah nilainya semakin baik
(C1_Stress Lvel, C2_Anxiety Score, C3_Depression Score, C4_Academic Pressure, C6_Screen Time, C9_Coding Hours, C10_Assignment Deadline)

	Kriteria	Bobot	Atribut
0	C1 (Stress Level)	0.1	Cost
1	C2 (Anxiety Score)	0.1	Cost
2	C3 (Depression Score)	0.1	Cost
3	C4 (Academic Pressure)	0.1	Cost
4	C5 (Social Support)	0.1	Benefit
5	C6 (Screen Time)	0.1	Cost
6	C7 (Sleep Quality)	0.1	Benefit
7	C8 (Daily Sleep Hours)	0.1	Benefit
8	C9 (Coding Hours)	0.1	Cost
9	C10 (Assignment Deadline)	0.1	Cost


```

v ATRIBUT KRITERIA

[41]
✓ 0d

# =====
# ATRIBUT KRITERIA
# True = Benefit
# False = Cost
# =====

is_benefit = [
    False, # stress
    False, # anxiety
    False, # depression
    False, # academic pressure
    True, # social support
    False, # screen time
    True, # sleep quality
    True, # sleep hours
    False, # coding hours
    False # assignment deadline
]

# =====
# TABEL KRITERIA
# =====

kriteria_names = [
    'C1 (Stress Level)',
    'C2 (Anxiety Score)',
    'C3 (Depression Score)',
    'C4 (Academic Pressure)',
    'C5 (Social Support)',
    'C6 (Screen Time)',
    'C7 (Sleep Quality)',
    'C8 (Daily Sleep Hours)',
    'C9 (Coding Hours)',
    'C10 (Assignment Deadline)'
]

df_kriteria = pd.DataFrame({
    'Kriteria': kriteria_names,
    'Bobot': weights,
    'Atribut': [
        'Benefit' if b else 'Cost'
        for b in is_benefit
    ]
})

display(df_kriteria)

```

	Kriteria	Bobot	Atribut
0	C1 (Stress Level)	0.1	Cost
1	C2 (Anxiety Score)	0.1	Cost
2	C3 (Depression Score)	0.1	Cost
3	C4 (Academic Pressure)	0.1	Cost
4	C5 (Social Support)	0.1	Benefit
5	C6 (Screen Time)	0.1	Cost
6	C7 (Sleep Quality)	0.1	Benefit
7	C8 (Daily Sleep Hours)	0.1	Benefit
8	C9 (Coding Hours)	0.1	Cost
9	C10 (Assignment Deadline)	0.1	Cost

4. Normalisasi SAW

Normalisasikan dilakukan untuk menyamakan skala antar kriteria menggunakan rumus:

- Benefit : $r_{ij} = x_{ij} / \max(x_j)$ Nilai dibagi nilai maksimum kolom
- Cost : $r_{ij} = \min(x_{ij}) / x_{ij}$ Nilai minimum kolom dibagi nilai tiap baris

NORMALISASI SAW

```
# =====
# NORMALISASI SAW
# =====

# mengambil matrix angka
matrix = df_saw.iloc[:, 1:].values

# membuat matrix kosong
normalized_matrix = np.zeros(matrix.shape)

# =====
# PROSES NORMALISASI
# =====

for j in range(matrix.shape[1]):

    if is_benefit[j]:

        # benefit
        normalized_matrix[:, j] = (
            matrix[:, j] /
            np.max(matrix[:, j])
        )

    else:

        # cost
        normalized_matrix[:, j] = (
            np.min(matrix[:, j]) /
            matrix[:, j]
        )

# =====
# HASIL NORMALISASI
# =====

df_norm = pd.DataFrame(
    normalized_matrix,
    columns=df_saw.columns[1:],
    index=df_saw['Alternatif']
)

print("=== MATRIX NORMALISASI ===")

display(df_norm)
```

... === MATRIX NORMALISASI ===

	C1 (Stress Level)	C2 (Anxiety Score)	C3 (Depression Score)	C4 (Academic Pressure)	C5 (Social Support)	C6 (Screen Time)	C7 (Sleep Quality)	C8 (Daily Sleep Hours)	C9 (Coding Hours)	C10 (Assignment Deadline)
Alternatif										
100001	0.333333	0.100000	0.333333	0.750000	0.666667	0.213115	0.666667	0.8500	0.348387	0.333333
100002	0.333333	0.500000	0.100000	0.375000	1.000000	0.433333	0.333333	0.5875	0.159343	0.166667
100003	1.000000	0.500000	0.142857	0.375000	0.333333	0.866667	1.000000	0.6000	0.200680	0.500000
100004	0.333333	0.333333	0.333333	0.750000	1.000000	0.185714	0.333333	0.8375	0.238380	0.125000
100005	0.333333	0.142857	0.142857	0.500000	0.444444	0.240741	1.000000	0.8375	0.633380	0.250000
100006	0.333333	0.125000	0.250000	0.333333	0.222222	0.317073	0.333333	0.6000	0.633437	0.100000
100007	1.000000	0.200000	0.111111	0.333333	0.666667	0.166667	0.666667	1.0000	1.000000	0.125000
100008	0.500000	0.200000	0.100000	1.000000	0.666667	0.171053	0.333333	0.7875	0.173127	0.125000
100009	1.000000	1.000000	1.000000	0.428571	0.111111	1.000000	0.666667	0.6750	0.237558	0.125000
100010	0.500000	0.250000	0.500000	0.750000	0.777778	0.113043	0.333333	0.8750	0.206541	1.000000

5. Perhitungan SAW

Skor akhir dihitung dengan mengalikan nilai ternormalisasi dengan bobot masing-masing kriteria kemudian dijumlahkan : $V_i = \sum (w_j \times r_{ij})$. Proses ini dilakukan menggunakan np.dot() yaitu perkalian matriks antara normalized_matrix dan weights. Kemudian melakukan penanganan untuk nilai NaN dan inf jika terdapat

nilai 0 pada kolom Cost saat normalisasi ($\min/0 = \inf$). Nilai tersebut diganti dengan 0 agar tidak mengganggu proses ranking. Alternatif diurutkan berdasarkan skor akhir dari tertinggi ke terendah. Skor tertinggi mendapat ranking 1 sebagai alternatif terbaik. Digunakan `method='dense'` agar tidak ada ranking yang terlewat jika terdapat nilai yang sama.

```
PERHITUNGAN SAW

# =====
# PERHITUNGAN SAW
# =====

# menghitung skor akhir
skor_akhir = np.dot(
    normalized_matrix,
    weights
)

# simpan skor akhir
df_saw['Skor_Akhir'] = skor_akhir

# =====
# MEMBERSIHKAN NILAI NaN DAN inf
# =====

df_saw = df_saw.replace(
    [np.inf, -np.inf],
    np.nan
)

df_saw = df_saw.fillna(0)

# =====
# CEK SKOR AKHIR
# =====

print(df_saw['Skor_Akhir'])

# =====
# RANKING
# =====

ranking_values = (
    df_saw['Skor_Akhir']
    .rank(
        ascending=False,
        method='dense'
    )
)

# ubah NaN menjadi 0 dulu
ranking_values = ranking_values.fillna(0)

# convert integer
df_saw['Ranking'] = ranking_values.astype(int)

# =====
# HASIL AKHIR
# =====
```

```

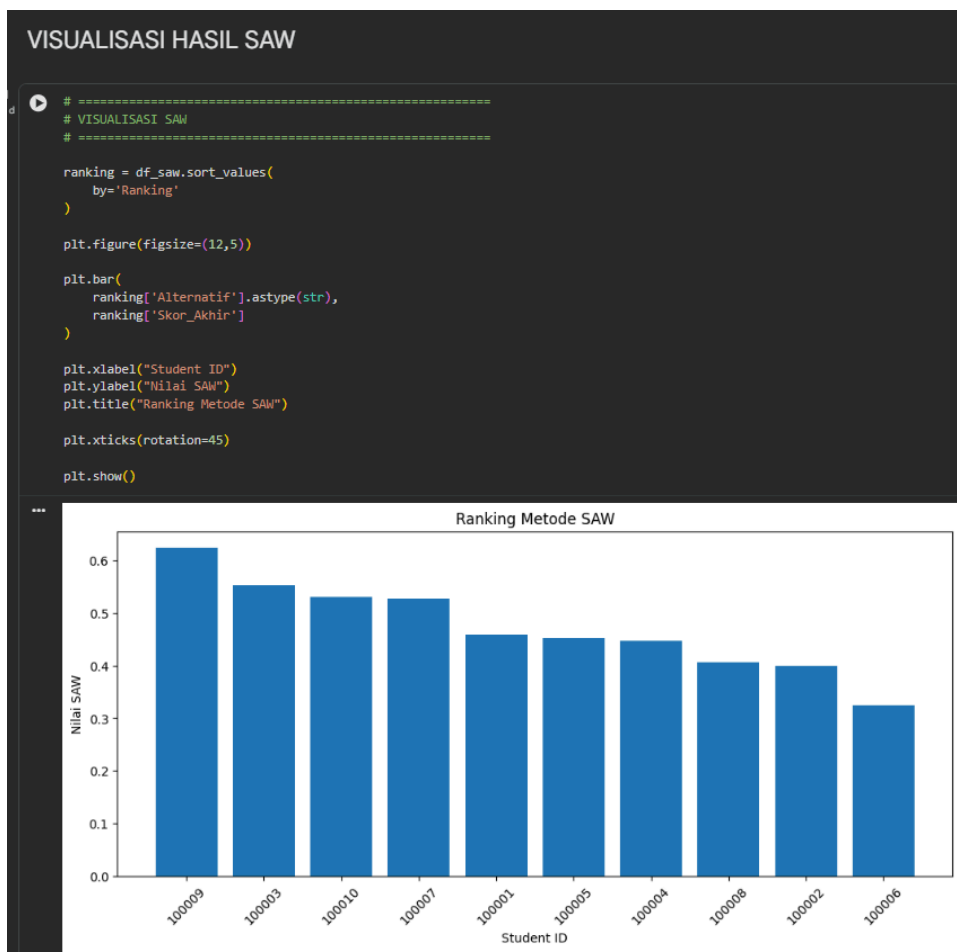
... 0 0.459484
1 0.398851
2 0.551854
3 0.446893
4 0.452511
5 0.324773
6 0.526944
7 0.405668
8 0.624391
9 0.530570
Name: Skor_Akhir, dtype: float64
=== HASIL AKHIR SAW ===

```

		C1 (Stress Level)	C2 (Anxiety Score)	C3 (Depression Score)	C4 (Academic Pressure)	C5 (Social Support)	C6 (Screen Time)	C7 (Sleep Quality)	C8 (Daily Sleep Hours)	C9 (Coding Hours)	C10 (Assignment Deadline)	Skor_Akh
8	100009	1	1	1	7	1	1.3	2	5.4	6.410035	8	0.6243
2	100003	1	2	7	8	3	1.5	3	4.8	7.587945	2	0.5518
9	100010	2	4	2	4	7	11.5	1	7.0	7.372653	1	0.5305
6	100007	1	5	9	9	6	7.8	2	8.0	1.522753	8	0.5269
0	100001	3	10	3	4	6	6.1	2	6.8	4.370861	3	0.4594
4	100005	3	7	7	6	4	5.4	3	6.7	2.404168	4	0.4525
3	100004	3	3	3	4	9	7.0	1	6.7	6.387926	8	0.4468
7	100008	2	5	10	3	6	7.6	1	6.3	8.795585	8	0.4056
1	100002	3	2	10	8	9	3.0	1	4.7	9.556429	6	0.3988
5	100006	3	8	4	9	2	4.1	1	4.8	2.403951	10	0.3247

6. Visualisasi Hasil SAW

Hasil ranking divisualisasikan dalam bentuk bar chart dengan sumbu X berisi student_id dan sumbu Y berisi nilai SAW. Visualisasi ini memudahkan pembacaan hasil ranking secara visual.



3.2 Metode EDAS

Metode EDAS (*Evaluation based on Distance from Average Solution*) adalah metode pengambilan keputusan multi-kriteria (MCDM) yang mengevaluasi setiap alternatif berdasarkan jaraknya terhadap nilai rata-rata (*Average Solution*) dari setiap kriteria.

Google Collab :

https://colab.research.google.com/drive/1B1Lkh5egI2p40j3g_GdLaX5K9J3n-X3?usp=sharing#scrollTo=AXDgcQ_bjEwy

Tahapan metode SAW meliputi:

1. Menentukan Kriteria

Definisikan 10 kriteria (C1 - C10) beserta tipe dan bobotnya. Bobot setiap kriteria adalah 0.10 dengan total bobot = 1.0. Diambil 10 data pertama dari dataset sebagai alternatif dengan format penamaan student_id (A1 - A10).

```
1) # ===== MENENTUKAN Kriteria =====
# STEP 1 - DETERMINING CRITERIA & WEIGHTS
# =====

# Kriteria = [
#   C1 (Stress Level), C2 (Anxiety Score), C3 (Depression Score),
#   C4 (Academic Pressure), C5 (Social Support), C6 (Screen Time),
#   C7 (Sleep Quality), C8 (Daily Sleep Hours),
#   C9 (Coding Hours), C10 (Assignment Deadline)
# ]

types = ['Cost', 'Cost', 'Cost', 'Cost', 'Benefit', 'Cost', 'Benefit', 'Benefit', 'Benefit', 'Cost']

weights = np.array([0.10]*10)
print("Total Bobot: ", weights.sum())

# Data: df_kriteria, Aturan, dan Bobot ---
df_kriteria = pd.DataFrame(
    {
        'Kriteria': criteria,
        'Tipe': types,
        'Bobot': weights
    }
)
print("\n--- 1. Label Kriteria, Aturan, dan Bobot ---")
display(df_kriteria)

# Data: Label Data Mentah ---
kriteria_data = {
    'stress_level_score': 'anxiety_score', 'depression_score',
    'academic_pressure_score', 'social_support_score', 'screen_time_hours',
    'sleep_quality_score', 'daily_sleep_hours',
    'coding_hours_per_day', 'assignment_deadline_percent'
}

student_id = df_kriteria['id'].head(10).values
alternatif = [f"A{i+1}" for i, id in enumerate(student_id)]

df_1 = pd.DataFrame(
    df_kriteria[['Kriteria', 'Tipe', 'Bobot']].values,
    index=alternatif,
    columns=kriteria
)
print("\n--- 2. Label Data Mentah ---")
display(df_1)
```

2. Menghitung Average Solution (AV)

Hitung nilai rata-rata setiap kriteria dari seluruh alternatif menggunakan `df_X.mean()`. Nilai AV menjadi acuan utama dalam metode EDAS untuk mengukur seberapa jauh alternatif berada diatas atau dibawah rata-rata dengan rumus : $AV_j = (1/n) \times \sum X_{ij}$.

```
1) # ===== RATA-RATA (AV) =====
# STEP 3 - AVERAGE SOLUTION (AV)
# =====

AV = df_X.mean()

print("\n--- 2. Average Solution (AV) ---")
display(pd.DataFrame(AV, columns=['Nilai Rata-rata (AV)'], T))

--- 2. Average Solution (AV) ---

```

	C1 (Stress Level)	C2 (Anxiety Score)	C3 (Depression Score)	C4 (Academic Pressure)	C5 (Social Support)	C6 (Screen Time)	C7 (Sleep Quality)	C8 (Daily Sleep Hours)	C9 (Coding Hours)	C10 (Assignment Deadline)
Nilai Rata-rata (AV)	2.2	4.7	5.0	0.2	5.3	5.53	1.7	8.12	5.681231	5.8

3. Menghitung PDA dan NDA

Kemudian hitung jarak setiap alternatif terhadap nilai rata-rata:

Kriteria benefit:

- $PDA = \max(0, (X - AV)) / AV \rightarrow$ seberapa jauh di Atas rata-rata
- $NDA = \max(0, (AV - X)) / AV \rightarrow$ seberapa jauh di Bawah rata-rata

Kriteria Cost:

- $PDA = \max(0, (X - AV)) / AV \rightarrow$ nilai lebih kecil dari rata-rata = baik
- $NDA = \max(0, (AV - X)) / AV \rightarrow$ nilai lebih besar dari rata-rata = buruk

Nilai $\max(0, \dots)$ memastikan tidak ada jarak negatif.

- PDA tinggi = alternatif lebih baik dari rata-rata
- NDA tinggi = alternatif lebih buruk dari rata-rata

```

# -----
# STEP 2 - PDA DAN NDA
# -----

PDA = pd.DataFrame(np.zeros(df_X.shape), index=alternatives, columns=criteria)
NDA = pd.DataFrame(np.zeros(df_X.shape), index=alternatives, columns=criteria)

for j, c in enumerate(criteria):
    if types[j].lower() == 'benefit':
        PDA[c] = np.maximum(0, (df_X[c] - AV[c]) / AV[c])
        NDA[c] = np.maximum(0, (AV[c] - df_X[c]) / AV[c])
    elif types[j].lower() == 'cost':
        PDA[c] = np.maximum(0, (AV[c] - df_X[c]) / AV[c])
        NDA[c] = np.maximum(0, (df_X[c] - AV[c]) / AV[c])

print("\n--- 3a. Positive Distance from Average (PDA) ---")
display(PDA)
print("\n--- 3b. Negative Distance from Average (NDA) ---")
display(NDA)

```

	C1 (Stress Level)	C2 (Anxiety Score)	C3 (Depression Score)	C4 (Academic Pressure)	C5 (Social Support)	C6 (Screen Time)	C7 (Sleep Quality)	C8 (Daily Sleep Hours)	C9 (Coding Hours)	C10 (Assignment Deadline)
100001 (A1)	0.000000	0.000000	0.454288	0.354839	0.132075	0.000000	0.176471	0.111111	0.000000	0.422759
100002 (A2)	0.000000	0.574468	0.000000	0.000000	0.898113	0.457505	0.000000	0.000000	0.682105	0.000000
100003 (A3)	0.545455	0.574468	0.000000	0.000000	0.000000	0.728752	0.784706	0.000000	0.335917	0.855172
100004 (A4)	0.000000	0.381702	0.454288	0.354839	0.898113	0.000000	0.000000	0.094771	0.124391	0.000000
100005 (A5)	0.000000	0.000000	0.000000	0.032298	0.000000	0.823508	0.784706	0.094771	0.000000	0.310345
100006 (A6)	0.000000	0.000000	0.285714	0.000000	0.000000	0.258590	0.000000	0.000000	0.000000	0.000000
100007 (A7)	0.545455	0.000000	0.000000	0.000000	0.132075	0.000000	0.176471	0.307190	0.000000	0.000000
100008 (A8)	0.090909	0.000000	0.000000	0.516129	0.132075	0.000000	0.000000	0.029412	0.548183	0.000000
100009 (A9)	0.545455	0.787234	0.000000	0.000000	0.000000	0.784919	0.176471	0.000000	0.128283	0.000000
100010 (A10)	0.000000	0.148939	0.842857	0.354839	0.320755	0.000000	0.000000	0.143701	0.297721	0.827588

4. Menghitung SP dan SN

Selanjutnya hitung jumlah terbobot dari PDA dan NDA dengan mengalikan setiap nilai dengan bobot kriteria:

- $SP_i = \sum (w_j \times PDA_{ij}) \rightarrow$ semakin tinggi semakin baik
- $SN_i = \sum (w_j \times NDA_{ij}) \rightarrow$ semakin rendah semakin baik

SP merepresentasikan total kelebihan alternatif di atas rata-rata, sedangkan SN merepresentasikan total kekurangan alternatif di bawah rata-rata.

```

# -----
# STEP 4 - SP DAN SN
# -----

SP = (PDA * weights).sum(axis=1)
SN = (NDA * weights).sum(axis=1)

df_5 = pd.DataFrame({'SP (Sum of PDA)': SP, 'SN (Sum of NDA)': SN})
print("\n--- 4. Jumlah Terbobot SP & SN ---")
display(df_5)

```

	SP (Sum of PDA)	SN (Sum of NDA)
100001 (A1)	0.172154	0.182502
100002 (A2)	0.241219	0.211795
100003 (A3)	0.380417	0.118997
100004 (A4)	0.200910	0.142053
100005 (A5)	0.122559	0.182510
100006 (A6)	0.054430	0.408847
100007 (A7)	0.110119	0.264435
100008 (A8)	0.131071	0.201494
100009 (A9)	0.322379	0.143731
100010 (A10)	0.282739	0.149133

5. Normalisasi NSP dan NSN

Setelah melakukan perhitungan SP dan SN normalisasikan agar nilai berada pada skala 0 - 1:

- $NSP_i = SP_i / \max(SP)$

$$b. \text{NSN}_i = 1 - (\text{SN}_i / \max(\text{SN}))$$

NSP mendekati 1 berarti alternatif jauh di atas rata-rata. sedangkan NSN mendekati berarti alternatif tidak jauh di bawah rata-rata.

```

# =====
# STEP 5 - NORMALISASI NSP DAN NSN
# =====

NSP = SP / SP_max()
NSN = 1 - (SN / SN_max())

df_NS = pd.DataFrame({'NSP': NSP, 'NSN': NSN})
print("\n--- 5. Normalisasi NSP & NSN ---")
display(df_NS)

```

	NSP	NSN
100001 (A1)	0.477652	0.551424
100002 (A2)	0.886278	0.479424
100003 (A3)	1.000000	0.707814
100004 (A4)	0.582132	0.850843
100005 (A5)	0.340047	0.520823
100006 (A6)	0.151021	0.000000
100007 (A7)	0.322180	0.350037
100008 (A8)	0.385320	0.504742
100009 (A9)	0.894401	0.849720
100010 (A10)	0.784470	0.833442

6. Appraisal Score dan Rangking

Setelah mendapatkan nilai NSP dan NSN langkah selanjutnya adalah menghitung skor akhir (AS sebagai rata-rata dari NSP dan NSN dengan rumus : $AS_i = 0.5 \times (NSP_i + NSN_i)$). Hasil akhir perhitungan EDAS berupa nilai appraisal score dan ranking mahasiswa berdasarkan tingkat burnout. Semakin besar nilai appraisal score maka alternatif dianggap lebih baik sesuai tujuan analisis.

```

# =====
# STEP 6 - APPRAISAL SCORE & RANKING
# =====

AS = 0.5 * (NSP + NSN)

df_final = pd.DataFrame({'Appraisal Score (AS)': AS})
df_final['Peringkat'] = df_final['Appraisal Score (AS)'].rank(ascending=False).astype(int)
df_final = df_final.sort_values(by='Peringkat')

print("\n--- 6. Nilai Skor Penilaian (AS) dan Perangkingan Akhir ---")
display(df_final)

```

	Appraisal Score (AS)	Peringkat
100003 (A3)	0.853757	1
100009 (A9)	0.770590	2
100010 (A10)	0.708000	3
100004 (A4)	0.619487	4
100002 (A2)	0.574351	5
100001 (A1)	0.514538	6
100008 (A8)	0.435038	7
100005 (A5)	0.433435	8
100007 (A7)	0.336109	9
100006 (A6)	0.075510	10

7. Visualisasi Hasil EDAS

Visualisasi menunjukkan hasil ranking burnout mahasiswa menggunakan metode EDAS dalam bentuk grafik atau diagram agar lebih mudah dianalisis. Alternatif diurutkan berdasarkan peringkat dari tertinggi ke terendah untuk memudahkan pembacaan hasil keputusan secara visual.



BAB 4 ANALISIS & PERBANDINGAN MODEL

4.1 Hasil Metode SAW

Alternatif	C1 (Stress Level)	C2 (Anxiety Score)	C3 (Depression Score)	C4 (Academic Pressure)	C5 (Social Support)	C6 (Screen Time)	C7 (Sleep Quality)	C8 (Daily Sleep Hours)	C9 (Coding Hours)	C10 (Assignment Deadline)	Skor_Akhir	Ranking
8 100009	1	1	1	7	1	1.3	2	5.4	6.410035	8	0.624391	1
2 100003	1	2	7	8	3	1.5	3	4.8	7.587945	2	0.551854	2
9 100010	2	4	2	4	7	11.5	1	7.0	7.372653	1	0.530570	3
6 100007	1	5	9	9	6	7.8	2	8.0	1.522753	8	0.526944	4
0 100001	3	10	3	4	6	6.1	2	6.8	4.370861	3	0.459484	5
4 100005	3	7	7	6	4	5.4	3	6.7	2.404188	4	0.452511	6
3 100004	3	3	3	4	9	7.0	1	6.7	6.387926	8	0.446993	7
7 100008	2	5	10	3	6	7.6	1	6.3	8.795585	8	0.405668	8
1 100002	3	2	10	8	9	3.0	1	4.7	9.556429	6	0.398851	9
5 100006	3	8	4	9	2	4.1	1	4.8	2.403951	10	0.324773	10

4.2 Hasil Metode EDAS

Alternatif	C1 (Stress Level)	C2 (Anxiety Score)	C3 (Depression Score)	C4 (Academic Pressure)	C5 (Social Support)	C6 (Screen Time)	C7 (Sleep Quality)	C8 (Daily Sleep Hours)	C9 (Coding Hours)	C10 (Assignment Deadline)	Appraisal Score	Ranking
2 100003 (A5)	1.0	2.0	7.0	8.0	3.0	1.5	3.0	4.8	7.587945	2.0	0.853757	1
8 100009 (A6)	1.0	1.0	1.0	7.0	1.0	1.3	2.0	5.4	6.410035	8.0	0.770560	2
9 100010 (A10)	2.0	4.0	2.0	4.0	7.0	11.5	1.0	7.0	7.372653	1.0	0.708960	3
3 100004 (A4)	3.0	3.0	3.0	4.0	9.0	7.0	1.0	6.7	6.387926	8.0	0.618487	4
1 100002 (A2)	3.0	2.0	10.0	8.0	9.0	3.0	1.0	4.7	9.556429	6.0	0.574351	5
0 100001 (A1)	3.0	10.0	3.0	4.0	6.0	6.1	2.0	6.8	4.370861	3.0	0.514538	6
7 100008 (A8)	2.0	5.0	10.0	3.0	6.0	7.6	1.0	6.3	8.795585	8.0	0.435030	7
4 100005 (A5)	3.0	7.0	7.0	6.0	4.0	5.4	3.0	6.7	2.404188	4.0	0.433435	8
6 100007 (A7)	1.0	5.0	9.0	9.0	8.0	7.8	2.0	8.0	1.522753	8.0	0.339109	9
5 100006 (A6)	3.0	8.0	4.0	9.0	2.0	4.1	1.0	4.8	2.403951	10.0	0.075610	10

4.3 Perbandingan Metode SAW dan EDAS

Google Colab :

https://colab.research.google.com/drive/1ChuvVEhn0m3PprzDwII0_ycMVo_99-rA?usp=sharing

4.3.1 Load Hasil SAW dan EDAS

Hasil perhitungan SAW dan EDAS disimpan dalam bentuk file csv, kemudian dari kedua dataframe gabungan menggunakan `pd.merge()` berdasarkan kolom Alternatif, kemudian diurutkan berdasarkan `Ranking_SAW`.

```
[7] # uploaded = files.upload() # pilih hasil_saw.csv DAN hasil_edas.csv
✓ Os
df_saw = pd.read_csv('/content/hasil_saw.csv')
df_edas = pd.read_csv('/content/hasil_edas.csv')

# Merge berdasarkan kolom Alternatif
df_perbandingan = pd.merge(df_saw, df_edas, on='Alternatif')
df_perbandingan = df_perbandingan.sort_values('Ranking_SAW').reset_index(drop=True)

print("=== PERBANDINGAN SAW vs EDAS ===")
display(df_perbandingan)
```

4.3.2 Analisis Perbandingan SAW dan EDAS

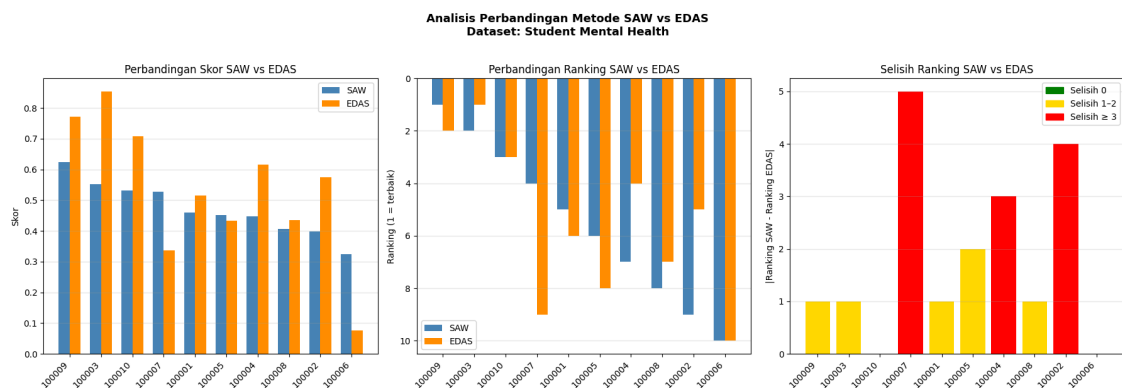
Aspek	SAW	EDAS
Alternatif Terbaik	100009	100003
Alternatif Terburuk	100006	100006
Rangking sama persis	2/10 alternatif	-

PERBANDINGAN SAW vs EDAS

Alternatif	Skor_SAW	Ranking_SAW	Skor_EDAS	Ranking_EDAS
0	100009	0.624391	1	0.770590
1	100003	0.551854	2	0.853757
2	100010	0.530570	3	0.708960
3	100007	0.526944	4	0.336109
4	100001	0.459484	5	0.514538
5	100005	0.452511	6	0.433435
6	100004	0.446993	7	0.616487
7	100008	0.405668	8	0.435036
8	100002	0.398851	9	0.574351
9	100006	0.324773	10	0.075510

Next steps: [Generate code with df_perbandingan](#) [New interactive sheet](#)

4.3.3 Visualisasi Perbandingan SAW dan EDAS



- Dari grafik diatas hasil dari Skor EDAS cenderung lebih tinggi dari SAW karena EDAS mengukur jarak dari rata-rata, bukan dari nilai maksimum.
- Perbandingan ranking SAW dan EDAS 100009 dan 100003 konsistensi di posisi teratas sedangkan 100006 konsistensi di posisi terakhir pada kedua metode
- Selisih ranking SAW dan EDAS terdapat 3 alternatif dengan selisih besar (merah) yaitu 100007 (selisih 5), 100002 (selisih 4), dan 100004 (selisih 3), memungkinkan ketiga alternatif tersebut dinilai berbeda oleh SAW dan EDAS.

4.4 Kesimpulan Perbandingan SAW dan EDAS

Hasil perbandingan SAW dan EDAS menunjukkan perbedaan pada alternatif terbaik, di mana SAW merekomendasikan 100009 dan EDAS 100003. Namun karena kedua metode sepakat bahwa 100009 berada di posisi dua teratas (SAW rank 1, EDAS rank 2) dan 100003 juga berada di dua teratas (SAW rank 2, EDAS rank 1) maka kedua mahasiswa tersebut sama-sama layak direkomendasikan sebagai alternatif terbaik. Sementara itu, 100006 disepakati sebagai alternatif terburuk oleh kedua metode.

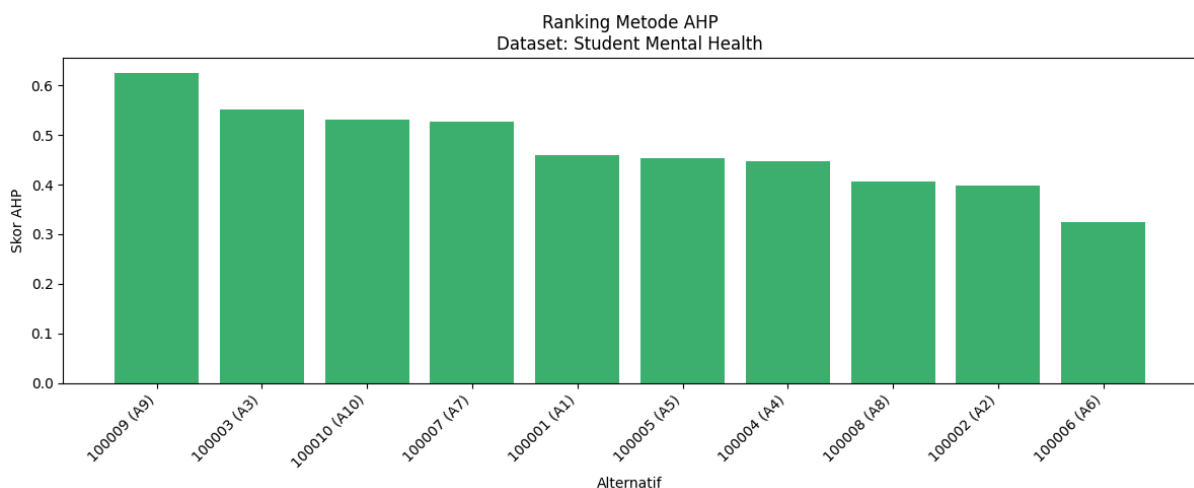
BAB 6 IMPLEMENTASI DAN PERBANDINGAN METODE

6.1 Implementasi Metode AHP

Analytical Hierarchy Process (AHP) adalah metode pengambilan keputusan yang menentukan bobot kriteria melalui perbandingan. Pada praktikum ini semua kriteria dianggap sama penting sehingga seluruh nilai perbandingan = 1 dan bobot tiap kriteria = 0.10.

Alternatif	Skor AHP	Ranking AHP
0 100009 (A9)	0.624391	1
1 100003 (A3)	0.551854	2
2 100010 (A10)	0.530570	3
3 100007 (A7)	0.526944	4
4 100001 (A1)	0.459484	5
5 100005 (A5)	0.452511	6
6 100004 (A4)	0.446993	7
7 100008 (A8)	0.405668	8
8 100002 (A2)	0.398851	9
9 100006 (A6)	0.324773	10

(Hasil Skor AHP)



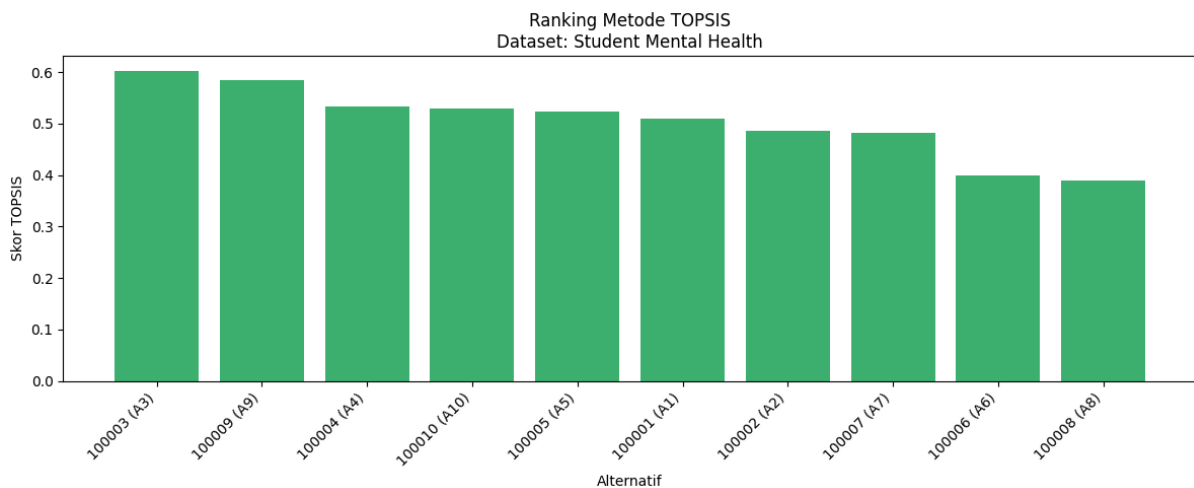
(Visualisasi AHP)

6.2 Implementasi Metode TOPSIS

Technique for Order Preference by Similarity to Ideal Solution (TOPSIS) adalah metode yang memilih alternatif terbaik berdasarkan jarak terdekat ke solusi ideal (A^+) dan terjauh dari solusi ideal negatif (A^-). Skor dihitung dengan rumus $V = D^- / (D^+ + D^-)$ dimana skor mendekati 1 berarti alternatif terbaik.

	Alternatif	D+	D-	Skor TOPSIS	Ranking TOPSIS
0	100003 (A3)	0.061035	0.092225	0.601755	1
1	100009 (A9)	0.066559	0.093504	0.584170	2
2	100004 (A4)	0.068808	0.078729	0.533621	3
3	100010 (A10)	0.072634	0.081909	0.529322	4
4	100005 (A5)	0.066434	0.072920	0.523268	5
5	100001 (A1)	0.070873	0.073788	0.510074	6
6	100002 (A2)	0.083586	0.078639	0.485387	7
7	100007 (A7)	0.075332	0.070260	0.482583	8
8	100006 (A6)	0.091444	0.060838	0.399510	9
9	100008 (A8)	0.087359	0.055945	0.390394	10

(Hasil Skor Topsis)



(Visualisasi Topsis)

6.3 Perbandingan Keempat Metode

Berdasarkan hasil perbandingan keempat metode, 100009 (A9) dan 100003 (A3) secara konsisten berada di dua posisi teratas di semua metode, SAW dan AHP menempatkan 100009 pada posisi terbaik, sementara EDAS dan TOPSIS menempatkan 100003 sebagai posisi terbaik.

TABEL PERBANDINGAN 4 METODE									
	Alternatif	Skor SAW	Ranking SAW	Skor EDAS	Ranking EDAS	Skor AHP	Ranking AHP	Skor TOPSIS	Ranking TOPSIS
0	100009 (A9)	0.624391	1	0.770590	2	0.624391	1	0.584170	2
1	100003 (A3)	0.551854	2	0.853757	1	0.551854	2	0.601755	1
2	100010 (A10)	0.530570	3	0.708960	3	0.530570	3	0.529322	4
3	100007 (A7)	0.526944	4	0.336109	9	0.526944	4	0.482583	8
4	100001 (A1)	0.459484	5	0.514538	6	0.459484	5	0.510074	6
5	100005 (A5)	0.452511	6	0.433435	8	0.452511	6	0.523268	5
6	100004 (A4)	0.446993	7	0.618487	4	0.446993	7	0.533621	3
7	100008 (A8)	0.405668	8	0.435036	7	0.405668	8	0.390394	10
8	100002 (A2)	0.388851	9	0.574351	5	0.388851	9	0.485387	7
9	100006 (A6)	0.324773	10	0.075510	10	0.324773	10	0.399510	9

Berdasarkan konsistensi SAW dan AHP menghasilkan ranking yang identik (10/10) dan EDAS dan TOPSIS memiliki konsistensi tertinggi berikutnya (3/10), maka 100009 (A9) direkomendasikan sebagai alternatif terbaik dengan kondisi kesehatan mental paling baik, dan 100006 (A6) disepakati sebagai alternatif terburuk oleh tiga dari empat metode.

```

[14] print("=== KONSISTENSI RANKING ===\n")
✓ On

metode = ['Ranking_SAW', 'Ranking_EDAS', 'Ranking_AHP', 'Ranking_TOPSIS']
pasangan = [('SAW', 'EDAS'), ('SAW', 'AHP'), ('SAW', 'TOPSIS'),
            ('EDAS', 'AHP'), ('EDAS', 'TOPSIS'), ('AHP', 'TOPSIS')]

for (m1, m2) in pasangan:
    sama = (df_all[f'Ranking_{m1}'] == df_all[f'Ranking_{m2}']).sum()
    print(f"{m1} vs {m2}: {sama}/10 alternatif ranking sama")

print("\n=== ALTERNATIF TERBAIK PER METODE ===")
for m in ['SAW', 'EDAS', 'AHP', 'TOPSIS']:
    terbaik = df_all.loc[df_all[f'Ranking_{m}'] == 1, 'Alternatif'].values[0]
    print(f"{m:6s}: {terbaik}")

```

```

▼ ... === KONSISTENSI RANKING ===

SAW vs EDAS: 2/10 alternatif ranking sama
SAW vs AHP: 10/10 alternatif ranking sama
SAW vs TOPSIS: 0/10 alternatif ranking sama
EDAS vs AHP: 2/10 alternatif ranking sama
EDAS vs TOPSIS: 3/10 alternatif ranking sama
AHP vs TOPSIS: 0/10 alternatif ranking sama

=== ALTERNATIF TERBAIK PER METODE ===
SAW      : 1000009 (A9)
EDAS     : 1000003 (A3)
AHP      : 1000009 (A9)
TOPSIS   : 1000003 (A3)

```

BAB 6 KESIMPULAN

6.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa sistem deteksi burnout mahasiswa berhasil dibuat menggunakan dataset Student Mental Health and Burnout Dataset. Proses preprocessing data dilakukan dengan baik melalui pengecekan missing value, konversi data kategorikal, dan penambahan variabel simulasi agar data dapat digunakan dalam proses perhitungan.

Metode SAW digunakan dengan perhitungan normalisasi dan penjumlahan nilai berbobot sehingga menghasilkan ranking alternatif secara sederhana dan mudah dipahami. Metode EDAS menggunakan perbandingan terhadap nilai rata-rata untuk melihat jarak positif dan negatif setiap alternatif, sehingga hasil analisis lebih stabil dan detail. Metode AHP digunakan untuk menentukan bobot kriteria secara sistematis, sedangkan TOPSIS menentukan alternatif terbaik berdasarkan kedekatan dengan solusi ideal.

Berdasarkan hasil analisis, keempat metode menunjukkan bahwa alternatif 100009 (A9) dan 100003 (A3) selalu berada pada posisi terbaik di semua metode. SAW dan AHP menghasilkan ranking yang identik dengan tingkat konsistensi tertinggi, sedangkan EDAS dan TOPSIS juga menunjukkan hasil yang hampir serupa. Oleh karena itu, 100009 (A9) direkomendasikan sebagai alternatif terbaik, sedangkan 100006 (A6) menjadi alternatif terlemah.

6.2 Saran

1. Sistem dapat dikembangkan menjadi aplikasi berbasis web atau mobile agar dapat digunakan secara langsung oleh mahasiswa.
2. Variabel penelitian dapat diperluas dengan menambahkan faktor lain seperti aktivitas organisasi, kondisi ekonomi, dan lingkungan sosial mahasiswa.
3. Visualisasi hasil dapat dibuat lebih interaktif agar pengguna mudah memahami tingkat burnout yang dialami.

LAMPIRAN

Link Repositori Github	https://github.com/EksaMuhammad/UTS-SBP-Kel-5/tree/main/kuis2
Link Google Collab SAW	https://colab.research.google.com/drive/10wxW9670bQO8ausLqOmjCPNZfxfvK7ef?usp=sharing
Link Google Collab EDAS	https://colab.research.google.com/drive/1B1Lkh5egII2p40j3g_GdLaX5K9J3n-X3?usp=sharing#scrollTo=AXDgcQ_bjEwv
Link Google Collab Perbandingan	https://colab.research.google.com/drive/1ChuvVEhn0m3PprzDwlI0_ycMVo_99-rA?usp=sharing
Link Dataset	https://www.kaggle.com/datasets/sehaj1104/student-mental-health-and-burnout-dataset